

必剪Android项目组件化最佳实践



哔哩哔哩技术

编辑于 2022年10月14日 14:30

...

本期作者



王良河

B端技术中心资深开发工程师

01 前言

必剪以前的代码架构是单一工程模式，对于小型产品为了快速迭代第一个版本上线，这种模式是没问题的，但必剪的业务方向是专业剪辑平台，是重客户端型APP，业务和功能较多且复杂度很高。随着必剪项目快速发展，业务功能越来越多，代码量也越来越庞大，开发团队也在扩大，以前的项目代码架构和开发模式，存在较多的问题，已经不能满足业务快速迭代的诉求了。进而出现了开发效率、协作效率、编译效率等问题，具体问题如下：

1. 合代码经常发生冲突
2. 代码权限不可控，Code Review机制难以推行
3. 代码严重耦合，新人学习成本高，迭代和维护成本大，功能经常发生衰退
4. 增量编译比较耗时，影响开发效率
5. 组件无法实现平台化，比如海外版的剪辑APP、粉版创作模块和必剪有很多业务功能是一样的，类似的功能是否可以平台化，减少重复造轮子呢？
6. 项目分层模糊，基础功能修改，导致上层业务牵一发而动全身，测试部门担心功能衰退，需要全量测试，严重影响项目交付时间

02 名词解释

单一工程开发模式

一个代码工程(Project)对应一个APP，这个APP的所有业务功能都是集中在同一个工程里实现的，业务模块以package方式进行组织

模块化

编程领域的模块化,就是遵守代码设计七大原则,实现高内聚、低耦合、高复用,将统一模块的代码和资源放在一个module中。模块化相比于包来讲,模块化更灵活,类比于积木,我们可以用积木搭建出业务形态百变的APP

组件化

很多开发同学人都在问,模块化和组件化有什么区别?每个人的给出答案都不一样。个人的理解是区别其实很小,但又并不是完全相同的概念。通过以上模块化的概念讲述,大家应该对模块化有了一个整体的了解,那么具体区别是什么呢?组件化是建立在模块化思想上演进的一个变种。组件化本来就是模块化的概念。组件化的核心是角色的转换。在打包时是library,在调试时是application(单独的app)

组件既可以是业务组件也可以是功能组件。以前的业务模块化演变成组件化,最核心的变化是就在于library和application的切换。比如你对某个业务模块进行了组件化,如果是集成调试(library模式)时,该业务模块就是一个common module或者是aar包。单独调试(application模式)时,该业务模块就是单独的app。在调试阶段,我只关心我负责的模块,我希望我的模块是一个单独的app,因为这样更小,业务更专一,职责边界清晰,修改与调试代码就会越省时省心,编译也会大幅提效。试想当你需要改一段代码,既要关注自己的,也要关注别人的,是一种什么体验?当项目足够大,业务线多达几十个,每个业务线都是隔离的,当业务线之间的开发是有相互依赖的,当出现问题,沟通成本是巨大的。编译一个10M的工程代码和1G的工程代码,编译耗时差的不是一个量级。

03 单一工程开发模式痛点

编译效率低效

APP开发,需要高频在手机进行调试,而每次调试都需要对整个工程进行编译,即便你只是改了一段代码或是UI调整,同样需要完整编译工程。当工程代码越来越大,编译也会越来越慢,每次增量编译,都需要等待4、5分钟,每个人一天平均编译时长高达1个小时,严重影响了开发效率。

增量构建的原理,是以工程目录为单位进行增量构建,发生变更时候,变更的工程,以及该工程作为父节点或祖先节点的工程(可以干预不构建,这也是快编的原理),均需要重新构建。如果是单一工程模式,增量编译维度就是整个项目,增量编译需要扫描该项目的所有代码的引用关系,扫描时长就会特别耗时

增量编译的还有一个关键因素是缓存命中范围,缓存命中范围越小,增量编译文件就会越多,时长就会越长。比如修改了某个基础库代码,由于项目代码没有模块化和组件化,依赖和调用该基础的代码,散在各个角落,这种情况下,就会导致项目中大部分代码都会被命中检索和重新编译

多人团队协作效率低效

每个小伙伴的开发任务虽然不同，但是都能修改整个工程的任意代码。为了完成自己的需求，团队内某个同学改了某段代码，但是这个改动可能影响别人的功能，这样开发人员之间势必要花更多的时间去沟通和协调，没法专注自己的功能点。十几个人同时合并代码时，代码会出现大量的冲突，解决冲突需要耗费巨大的人力且有可能随时合出问题或合漏代码，而且必剪的合代码规则是，产品验收通过之后，代码才能合入至主分支。我们团队周三下午7点出总包，下午5点才开始陆陆续续合代码，如果合代码出现问题，就会影响出总包，进而导致不能按时交付

组件化&模块化功能复用基石

必剪是一个专业剪辑平台，编辑功能比较丰富和专业。B站同时也有一些部门在做剪辑相关的功能和业务，必剪不仅要提解决团队内部迭代问题。从长远的发展来看，必剪需要输出平台化的剪辑业务组件、基础功能组件，赋能至别的部门，减少重复造轮子，必剪于2022年7月给粉版输出了图文&UGC插件（业务平台级组件，和必剪共用同一组件），如果必剪没有组件化的基础，则无法做到这一点

很多公司对外宣称，三天可以产出一个APP，模块化和组件化起到了最核心的作用。组件就是积木，我们可以用积木搭建出业务形态百变的APP。我们只需要维护好每一个功能和业务组件，业务方一键集成即可

总结

以上提到3个痛点，并不代表只要是单一工程模式就一定会引发这些痛点，单一工程模式也可以通过自定义的管理工具（插件）和人为介入的方式进行规避。比如，单一工程方式以package方式组织模块代码和定义路径管理资源，然后通过自定义脚本指定对应的package和资源目录进行打aar包给别的部门复用。但如果这么做，开发成本维护成本都比较大，研发需要遵守的开发规范粒度就会变的更细更多，对于任何的自成体系的框架和系统，新人学习成本都比较高

04 组件化通用架构

以下示意图是个人理解的组件化通用架构，每个团队可以根据自身业务架构，进行对应的调整和扩展，比如基础业务层增加平台中间层，通过平台中间层的协议，实现平台业务组件，可以将业务组件复用给别的团队或部门



05 组件化开发规范

规范1：动态配置组件工程类型

集成模式和组件单独调试模式，集成模式基本上是提测和上线发布的时候才会使用到，开发阶段建议使用组件单独调试模式，因为这样更小，业务更专一，职责边界清晰，修改与调试代码就会越省时省心，编译也会大幅提效。如何让组件在这两种调试模式之间自动转换呢？原理也比较简单，我们团队是通过提供AndroidStuido可视化界面插件，开发同学通过操作插件页面，一键切换集成模式和组件单独调试模式，由于篇幅有限，我就不在这里讲解AndroidStuido可视化界面插件的具体实现了。下面主要讲解动态配置组件工程类型的实现原理和规范：

- 1. 在工程目录，定义一个基础配置项的gradle文件，比如app_config.gradle,在该文件定义一个变量，标识哪个组件需要单独调试，debugModuleName = "";，当debugModuleName字段为空，则代表是集成模式，如果debugModuleName是某个组件名称，则代表是该组件需要独立调试。
- 2. 一个组件如果要独立运行，必须需要配置必备基础信息

2.1 apply " com.android.library "动态改成
"com.android.application"; 示例代码如下：

```
class InitApplyPluginTask extends InitBaseApplyPluginTask{  
  
    @Override  
    void input(Object ob) {  
        super.input(ob)  
    }  
  
    @Override  
    void doTask() {  
        if (PluginInfoManager.getInstance().getProject().getName().contains  
            initApplicationPlugin()  
        ){  
        }else{  
        }  
    }  
}
```

```

        initLibraryPlugin()
    }
}

@Override
Object output() {
    return super.output()
}

private void initApplicationPlugin() {
    PluginInfoManager.getInstance().getProject().apply plugin: "com

}

private void initLibraryPlugin() {
    PluginInfoManager.getInstance().getProject().apply plugin: "cc

}

}

```

2.2 一个App需要一个ApplicationId，而组件在独立调试时也是一个App，所以也需要一个ApplicationId，集成调试时组件是不需要ApplicationId的；另外一个APP也只有一个启动页，而组件在独立调试时也需要一个启动页，在集成调试时就不需要了。根据debugModuleName分别设置applicationId、AndroidManifest，示例代码如下：

```

class InitApplicationIdTask extends AbstractTask<Object> {
    @Override
    void input(Object ob) {
        super.input(ob)
    }

    @Override
    void doTask() {
        super.doTask()
        if (PluginInfoManager.getInstance().getProject().getName().contains(
            /**
             * 组件独立调试模式
             * 设置该module的applicationId
             */
            setApplicationId(PluginInfoManager.getInstance().getProject().ex
        )else{
            /**
             * 集成化模式，整个项目打包apk
             * 设置壳工程的applicationId
             */
            setApplicationId(PluginInfoManager.getInstance().getProject().ge
        }
    }

    @Override
    Object output() {
        return super.output()
    }

    private void setApplicationId(String applicationIdStr){
        PluginInfoManager.getInstance().getProject().android.defaultConfig {
            applicationId applicationIdStr
        }
    }
}

class InitSourceSetsTask extends AbstractTask<Object> {
    @Override
    void input(Object ob) {
        super.input(ob)
    }
}

```

```

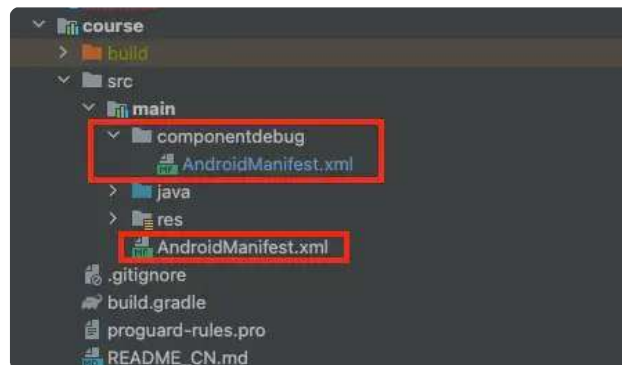
    }

    @Override
    void doTask() {
        super.doTask()
        if (PluginInfoManager.getInstance().getProject().getName().contains(
            PluginInfoManager.getInstance().getProject().android.sourceSe
                /**
                 * 组件独立调试模式
                 * 指定该module的AndroidManifest.xml
                 */
                manifest.srcFile &#39;src/main/componentdebug/Androi
            }
        }else{
            PluginInfoManager.getInstance().getProject().android.sourceSe
                /**
                 * 集成化模式，整个项目打包apk
                 * 指定壳工程的入口AndroidManifest.xml
                 */
                manifest.srcFile &#39;src/main/AndroidManifest.xml&#
            }
        }
    }

    @Override
    Object output() {
        return super.output()
    }
}

```

其中独立调试的AndroidManifest是新建于目录componentdebug，使用manifest.srcFile 即可指定两种调试模式的AndroidManifest文件路径。



componentdebug中新建的manifest文件指定了Application和启动activity, 示例代码如下:

```

<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">
    <package="com.bcut.course">
    <application
        android:name="com.bilibili.studio.module.course.CourseApplicati
    <activity
        android:name="BCutDebugMainActivity">
        <intent-filter>
            <action android:name="android.intent.action.MAIN" quc
            <category android:name="android.intent.category.LAUN
        </intent-filter>
    </activity>
    <activity
        android:name="com.bilibili.studio.module.course.ui.CourseVi

```

```
        android:screenOrientation="portrait" />
    </application>
</manifest>
```

BCutDebugMainActivity，所有业务模块的调试入口页面，业务方也可以自定义入口Activity

CourseApplication继承BaseApplication（负责基础组件的初始化），然后CourseApplication的onCreate生命周期初始化该业务module独自依赖的组件。如果是集成调试，组件生命周期分发管理器按照优先级配置项，给每个业务组件分发生命周期，然后每个业务组件在各自的Application完成初始化任务项。关于组件生命周期分发请见下文的“规范7：组件生命周期分发”章节。



规范2：组件间无直接依赖

组件化的核心就是**解耦**，所以组件间是不能有依赖的，只能依赖组件暴露服务的API，那么如何实现组件间的页面跳转和相互调用具体的实现类呢（**一个接口多个实现类**）？例如在首页模块需要账户信息或者需要跳转到主编辑器，两个模块之间没有依赖，也就说不能直接使用显式启动来打开主编辑器页面，那么隐式启动呢？隐式启动是可以实现跳转的，但是隐式Intent需要通过AndroidManifest配置和管理，协作开发比较麻烦，路由器实现的原理基本是一致的，都是基于Google的auto-service来实现APT（Annotation Processing Tool，**编译时注解处理器**）能力，然后编译时期自动生成和注入代码，为了和主站的技术方案保持一致，以及**组件平台化**，必剪项目使用主站提供的bilibiliRouter进行实现页面跳转、组件无直接依赖也可以相互调用等能力。通过Router的确是可以实现页面跳转、组件无直接依赖也可以相互调用，但组件内部的复杂业务耦合又如何解耦和简化代码呢，除了抽象协议和良好的代码设计模式，建议大家可以使用依赖注入框架比如hilt、koin等等，依赖项注入会为您的应用提供以下优势：

- 重用类以及分离依赖项**：更容易换掉依赖项的实现。由于控制反转，代码重用得以改进，并且类不再控制其依赖项的创建方式，而是支持任何配置。
- 易于重构**：依赖项成为 API Surface 的可验证部分，因此可以在创建对象时或编译时进行检查，而不是作为实现详情隐藏。
- 易于测试**：类不管理其依赖项，因此在测试时，您可以传入不同的实现以测试所有不同用例。

关于依赖注入框架使用和实现原理，有时间再写一篇技术文章对此进行深入讲解

规范3：资源命名规则

1. 所有资源文件的命名都需要以业务模块名为前缀，注意不要与其他业务模块前缀名冲突。假设业务模块名为`bgm`，则相关资源文件命名例子：

layout文件: bgm_activity_main.xml

anim文件: bgm_anim_bgm_list_detail_sheet_hide.xml

drawable文件: bgm_ic_audio_bgm_tryout.png

string: bgm_asr_recognize_lyric

2. 如果各种资源文件和定义，很多组件都使用到了，就要考虑下沉到公共组件了，每个公司的产品都应该会有固定的主题样式，包括颜色的定义、各种卡片默认图、按钮、弹框、theme等等，这些公共主题样式资源，需要下沉至公共组件。

一个庞大的商业项目中这么多资源文件，难道每个都要手动移动和重命名吗？显然是不行的，需要一个脚本批量处理，实现原理比较简单，代码量也不多，具体原理就不在这里阐述了，以下是源码：

```
import os
import sys
import re
import shutil

# 运行前先保证没有本地临时修改，方便revert

# 项目可配置项
# 目标module
target_module_name = "materialcenter"

# 项目根目录
project_path = "/Users/zhenglee/StudioProjects/bilibiliStudio/"

# 资源名前缀
prefix = "material_";

# 以下文件不要修改和缩进

# 遍历所有module路径
def ListModulePath(dirPath):
    fileList = []
    for root, dirs, files in os.walk(dirPath):
        for fileName in files:
            if fileName == "build.gradle" and not root.endswith(target_module_name):
                fileList.append(root)
    return fileList

origin_module_path = ListModulePath(project_path)

res_path = "src/main/res/"
targetPath = os.path.join(project_path, target_module_name, res_path)

code_path = "src/main/java/com/bilibili/studio/module/"
codeDir = os.path.join(project_path, target_module_name, code_path)

suffixs = [".xml", ".jpg", ".png", ".9.png", ".9"]

# 遍历文件夹
def ListFiles(dirPath):
    fileList = []
    for root, dirs, files in os.walk(dirPath):
        for fileObj in files:
```



```

        fileList.append(os.path.join(root, fileObj))
    return fileList

# 构造原始资源地址
def buildAddress(path,name):
    copyResult = False
    for modulePath in origin_module_path:
        originPath = os.path.join(modulePath,res_path)
        if os.path.exists(originPath):
            resDirs = os.listdir(originPath)
            for resDir in resDirs:
                if resDir.startswith(path):
                    origin = os.path.join(originPath,resDir,name)
                    target = os.path.join(targetPath,resDir,prefix+name)
                    copyResult =copyResult or copyFile(origin,target)
                    if copyResult:
                        return copyResult

    return copyResult

# 查找java文件中的关键字 (R.layout.xxxx) 并copy相应的资源
def FindInCodeAndCopy(filePath, regex):

    try:
        file_data = &quot;&quot;
        with open(filePath, &#39;r&#39;) as fileStream:

            times = 0
            for eachLine in fileStream:

                # 修改R文件引用
                eachLine = modifyFileStr(eachLine, &#39;import com.bilibili.

                # import kotlin.android.synthetic.main.material_layout_clou
                if eachLine.startswith(&#39;import kotlin.android.synthetic
                    eachLine = modifyFileStr(eachLine, &#39;import kotlin.a

                # 找出并过滤java中类似'R.layout.xxxx'的资源名称xxxx
                reg = r&#39;s(?:[\\n\\, , \\, \\;])&#39; %(regex)
                mat =re.findall(reg,eachLine,re.S)
                if len(mat)&gt;0:
                    for name in mat:
                        times = times+1;

                # 根据资源名称 name, 去资源目录找到相应资源并copy
                copyResult =buildAddress(regex.split(&#39;.&#39;)[1]

                # 修改资源名称以prefix开头
                if copyResult and not name.startswith(prefix):
                    eachLine = modifyFileStr(eachLine,name,prefix+na

            # 重写保存修改后的java文件内容
            file_data+=eachLine

        with open(filePath, &#39;w&#39;) as fileStream:
            # 重写写入java文件
            fileStream.write(file_data)

    except Exception as e:
        # print(e)
        fileStream.close()
    else:
        fileStream.close()
    return times

```

```

# 查找资源文件中的关键字 (&quot;@drawable/xxxx) 并copy相应的资源
def FindInResourceAndCopy(filePath, regex):

    try:
        file_data = &quot;&quot;
        with open(filePath, &#39;r&#39;) as fileStream:

            times = 0
            for eachLine in fileStream:

                # 找出并过滤java中类似'R.layout.xxxx'的资源名称xxxx
                reg = r&#39;%s(.*)[\&quot;]&#39; % (regex)
                mat = re.findall(reg, eachLine, re.S)
                if len(mat) > 0:
                    for name in mat:
                        times = times + 1;

                # 根据资源名称 name, 去资源目录找到相应资源并copy
                result = re.findall(&#39;[@](.*)[/]&#39;, regex)
                copyResult = buildAddress(result[0], name)

                # 修改资源名称以prefix开头
                if copyResult and not name.startswith(prefix):
                    eachLine = modifyFileStr(eachLine, name, prefix + name)

            # 重写保存修改后的java文件内容
            file_data += eachLine

        with open(filePath, &#39;w&#39;) as fileStream:
            # 重写写入java文件
            fileStream.write(file_data)

    except Exception as e:
        fileStream.close()
        print(filePath)
        print(e)
    else:
        fileStream.close()
    return times

# 修改文件内容
def modifyFileStr(line, old_str, new_str):
    if old_str in line:
        line = line.replace(old_str, new_str)
    return line

# copy文件
def copyFile(filepath, newpath):
    copyResult = False
    for suffix in suffixs:
        file_path = filepath + suffix
        new_path = newpath + suffix
        # print(&quot;file_path : %s      exists : %s&quot; % (file_path, os.
        # print(&quot;new_path : %s      exists : %s&quot; % (new_path, not(c

    if os.path.exists(file_path) and not(os.path.exists(new_path)):
        try:
            targetdir = os.path.dirname(new_path);
            if not os.path.exists(targetdir):
                os.makedirs(targetdir)
            shutil.copy(file_path, new_path)
        except Exception as e:
            print(&quot;copyFile error\nfile_path:%s\n      new_path:%s&quot; % (file_path, new_path)
            print(e)
        else:
            copyResult = True
            print(&quot;copyFile success\nfile_path:%s\n      new_path:%s&quot; % (file_path, new_path)
        finally:
            pass

```

```
return copyResult

def main():
    fileList = ListFiles(codeDir)
    times = 0
    for fileObj in fileList:
        for regex in [R.layout.'',R.drawable.'',R.'':
            times =times + FindInCodeAndCopy(fileObj, regex)

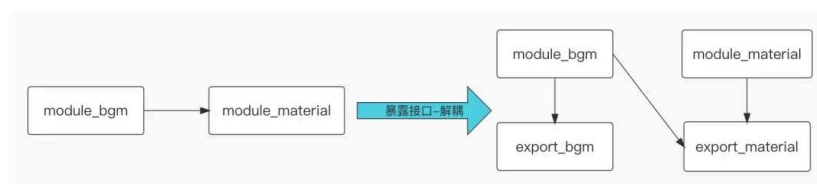
    # 遍历所有资源，找出并复制资源引用的资源
    resourcesList = ListFiles(targetPath)
    for resourceFile in resourcesList:
        if resourceFile.endswith(;'.xml;'):
            for resregex in [;'@drawable/;',;'@layout/;':
                FindInResourceAndCopy(resourceFile, resregex)

    print(;'代码中R.layout.、R.drawable.出现次数:%d;' %times)
    # os.system(;'pause;')

if __name__ == '__main__':
    main()
```

规范4：服务API暴露

平时开发中我们常用接口进行解耦，对接口的实现不用关心，避免接口调用与业务逻辑实现紧密关联。这里组件间的解耦也是相同的思路，仅依赖和调用服务接口，不会依赖接口的实现。可能你会疑问了，既然bgm组件可以访问material组件接口了，那就需要依赖material组件，这两个组件还是耦合了，这怎么处理呢？答案是**组件拆分出可暴露服务**。见下图：



左侧原有的代码结构，组件间调用对方服务，但是有依赖耦合。右侧多了**export_bgm、export_material**，这是对应拆分出来的专门用于提供服务的暴露组件。操作说明如下：

暴露组件，只存放服务接口、服务接口相关的实体类、路由信息、Fragment的获取等等

服务调用方只依赖服务提供方的报露组件，如module_bgm依赖export_material

组件需要依赖自己的暴露组件，并实现服务接口，如module_bgm依赖export_bgm并实现其中的服务接口

接口的实现注入由bilibiliRouter路由框架完成，和页面跳转一样使用路由信息

export_bgm示例代码:

```
interface IExportDemoService {
    val DemoServiceName: String?

    fun DemoToolsReset()

    fun DemoExtractNotifyDataChange()
}
```

export_bgm实现类代码:

```
@Services(IExportDemoService::class)
@Named("<ProxyDemoService>")
class ProxyDemoService : IExportDemoService {

    override val DemoServiceName: String
        get() = "<DemoModule_ProxyDemoService>"

    override fun DemoToolsReset() {
        DemoVideoExtractTools.getInst().reset()
    }

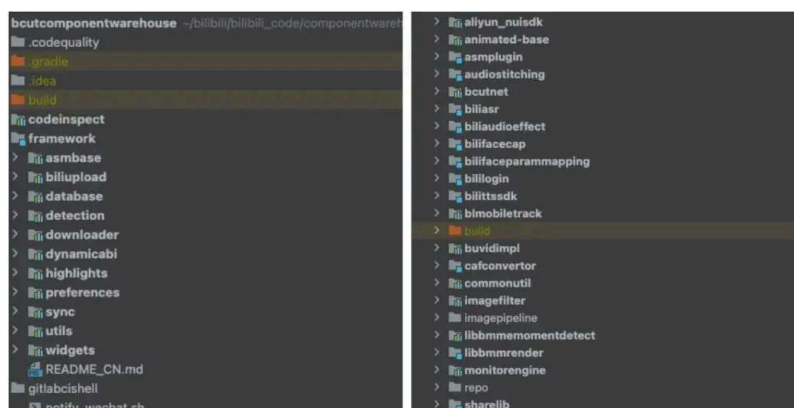
    override fun DemoExtractNotifyDataChange() {
        DemoVideoExtractTools.getInst().notifyDataChange()
    }
}
```

调用export_bgm示例代码:

```
BLRouter.INSTANCE.get(IExportDemoService.class, "<ProxyDemoService>")
```

规范5：第三方组件和自研基础功能组件

第三方组件和自研基础功能组件在多个业务组件中都有使用，需要这些底层组件库统一下沉和管理，下沉目的是跟业务隔离以及收敛修改权限，统一管理的目的 是规范和统一业务层依赖底层组件的版本。必剪自研组件的仓库示意图如下：



部分基础组件版本管理示意图：

```
maven_bcut_libraries = [
    'audiostitching' : "com.bcut.libs.libraries:audiostitching:${maven_bcut_libraries_versionMap.audiostitching}",
    'cafconverter'   : "com.bcut.libs.libraries:cafconverter:${maven_bcut_libraries_versionMap.cafconverter}",
    'libbmmomentdetect' : "com.bcut.libs.libraries:libbmmomentdetect:${maven_bcut_libraries_versionMap.libbmmomentdetect}",
    'animated-base'  : "com.bcut.libs.libraries:animated-base:${maven_bcut_libraries_versionMap.animated-base}",
    'imagepipeline'  : "com.bcut.libs.libraries:imagepipeline:${maven_bcut_libraries_versionMap.imagepipeline}",
    'blmobiletrack'  : "com.bcut.libs.libraries:blmobiletrack:${maven_bcut_libraries_versionMap.blmobiletrack}",
    'imagefilter'    : "com.bcut.libs.libraries:imagefilter:${maven_bcut_libraries_versionMap.imagefilter}",
    'monitor'        : "com.bcut.libs.libraries:monitor:${maven_bcut_libraries_versionMap.monitor}",
]

maven_bcut_framework = [
    'biliupload' : "com.bcut.libs.framework:biliupload:${maven_bcut_framework_versionMap.biliupload}",
    'database'   : "com.bcut.libs.framework:database:${maven_bcut_framework_versionMap.database}",
    'downloader' : "com.bcut.libs.framework:downloader:${maven_bcut_framework_versionMap.downloader}",
    'dynamicabi' : "com.bcut.libs.framework:dynamicabi:${maven_bcut_framework_versionMap.dynamicabi}",
    'highlights' : "com.bcut.libs.framework:highlights:${maven_bcut_framework_versionMap.highlights}",
    'preferences' : "com.bcut.libs.framework:preferences:${maven_bcut_framework_versionMap.preferences}",
    'detection'  : "com.bcut.libs.framework:detection:${maven_bcut_framework_versionMap.detection}"
]
```

规范6：数据存储

1. 使用SharedPreferences和MMKV时，每个业务模块只管理自己模块需要的数据，SharedPreferences文件名需要通过业务前缀来区分，MMKV实例（MMKV.mmkvWithID）需要通过业务前缀来区分，防止不同组件间数据发生冲突
2. 当某些数据需要全局共享时，可以考虑下沉到底层模块

规范7：组件生命周期分发

当项目组件化之后，业务组件之间是没有依赖的，每个组件都应该有独立的生命周期类似于Application，然后每个组件根据自己的业务和功能场景，在不同的生命周期回调做一些事情，比如组件依赖的基础库的初始化等等。那么如何做到各业务组件无侵入地获取 Application生命周期呢？原理也比较简单，主要分成四步：

第一步，定义组件的生命周期模型，每个组件实现生命周期模型，生命周期模型示例代码如下：

```
public enum LifecyclePriority {
    MIN_PRIORITY, NORM_PRIORITY, MAX_PRIORITY
}

public interface IApplicationLifecycle {
    LifecyclePriority getPriority();
    void onCreate(Context context);
    void onLowMemory();
    void onTrimMemory(int level);
}
```

第二步，生命周期管理类，示例代码如下：

```

public class ApplicationLifecycleManager {
    private static List<IApplicationLifecycle> APPLIFELIST = new ArrayList<>();
    private static boolean INITIALIZE = false;

    public static void init(Context context) {
        if (INITIALIZE){
            return;
        }
        INITIALIZE = true;
        loadModuleLife();
        Collections.sort(APPLIFELIST, new AppLikeComparator());
        for (IApplicationLifecycle appLife : APPLIFELIST) {
            appLife.onCreate(context);
        }
    }

    private static void loadModuleLife() {

    }

    private static void registerAppLife(String className) {
        if (TextUtils.isEmpty(className))
            return;
        try {
            Object obj = Class.forName(className).getConstructor().newInstance();
            if (obj instanceof IApplicationLifecycle) {
                APPLIFELIST.add((IApplicationLifecycle)obj);
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    private static void registerAppLife(IApplicationLifecycle appLife) {
        APPLIFELIST.add(appLife);
    }

    public static void onLowMemory() {
        for (IApplicationLifecycle appLife : APPLIFELIST) {
            appLife.onLowMemory();
        }
    }

    public static void onTrimMemory(int level) {
        for (IApplicationLifecycle appLife : APPLIFELIST) {
            appLife.onTrimMemory(level);
        }
    }

    static class AppLikeComparator implements Comparator<IApplicationLifecycle> {
        @Override
        public int compare(IApplicationLifecycle o1, IApplicationLifecycle o2) {
            int p1 = o1.getPriority().ordinal();
            int p2 = o2.getPriority().ordinal();
            return p2 - p1;
        }
    }
}

```

第三步，每个组件有诉求需要收到APP的生命周期，则需要实现 IApplicationLifecycle接口，示例代码如下：

```

@Services(value = IApplicationLifecycle.class)
@Named("&quot;HomePageApplication&quot;")
public class HomePageApplication implements IApplicationLifecycle {
    @Override
    public LifecyclePriority getPriority() {
        return LifecyclePriority.NORM_PRIORITY;
    }
    @Override
    public void onCreate(Context context) {

    }
    @Override
    public void onLowMemory() {

    }
    @Override
    public void onTrimMemory(int level) {

    }
}

```

第四步，前面三步完成之后，对于组件的生命周期绑定，有两种方式，一种硬代码编写，示例代码如下：

```

ArrayList<IApplicationLifecycle> moduleApplicationLifeList = new Array
moduleApplicationLifeList.add(BLRouter.INSTANCE.get(IApplicationLifecyc
moduleApplicationLifeList.add(BLRouter.INSTANCE.get(IApplicationLifecyc
moduleApplicationLifeList.add(BLRouter.INSTANCE.get(IApplicationLifecyc
ApplicationLifecycleManager.init(moduleApplicationLifeList);

```

这种方式扩展性不够且维护成本高，不同宿主可以根据不同的业务诉求，集成不同组件集，那每次都需要修改代码进行适配，成本太高了。第二种是自动注册，实现的原理也比较简单，主要是三个步骤：

第一个步骤：提前约定好使用APT生成每个组件ModuleApplication的代理类

第二个步骤：在jar包生成dex时，根据代理类约定好的类名规则，找到每个组件的ModuleApplication的代理类，写入至临时缓存

第三个步骤：最后找到ApplicationLifecycleManager，因为ApplicationLifecycleManager初始化的时候，会调用loadModuleLife，所以只要将对应的代码通过ASM插入至loadModuleLife方法即可。插桩后的示例如下：

```

private static void loadModuleLife() {
    registerAppLife("&quot;com.bilibili.studio.lifecycle.appt.HomePageApplic
    registerAppLife("&quot;com.bilibili.studio.lifecycle.appt.CourseApplicat
    registerAppLife("&quot;com.bilibili.studio.lifecycle.appt.GuiChuApplicat
}

```

规范8：开发文档

每个组件都要维护好说明文档，通常都是一个readme文件。一般包含以下说明：

- 组件的功能介绍
- 组件功能使用说明
- 组件历史版本记录
- 注意事项

尽量做到团队内任何一个成员，通过该文档就能使用组件，而不需要找到组件的开发人员来讲解。

06 Gradle配置统一管理

组件化开发之后，会尽量将所有业务模块根据业务和功能根据职责单一的边界进行拆分，这时候module数量会比较多，编译环境的不同的可能会导致编译结果的差异。举几个例子：当 `targetSdkVersion >= 23` 时，安卓引入了动态权限，敏感权限需要先申请再使用，但是`targetSdkVersion < 23` 时，是不需要申请的，如果有人使用了低版本 sdk，那么最终集成到主 app 中时，就可能会出现权限方面的问题了；其次就是支持的最小sdk 版本问题了，由于历史原因，很多 api 在高版本 sdk 中出现，如果有人的人在开发组件的过程中设置的 `minSdkVersion = 21`，但为了兼容更多的手机，集成打包时设置的 `minSdkVersion = 19`，那打包就会出现问或者是在低版本系统的手机上不兼容出现闪退。

必剪这边是自研了Gradle插件--GradleEngine来统一管理项目和所有组件的 **Gradle配置信息**，比如每个组件依赖的通用插件，BuildConfig注入编译环境和业务基础信息，Android系统相关SDK版本，APP主工程集成打包自动依赖所有业务组件等等，使用的方式也比较简单，对应module引用GradleEngine（`apply plugin: 'gradleengine'`）即可。只要组件依赖了GradleEngine，编译期就会自动给该组件完成注入配置信息。举个例子，编译时期GradleEngine调用`initBaseSdkVersionTask`完成AndroidSDK配置信息，示例代码如下：

```
class InitBaseSdkVersionTask extends AbstractTask<Object> {

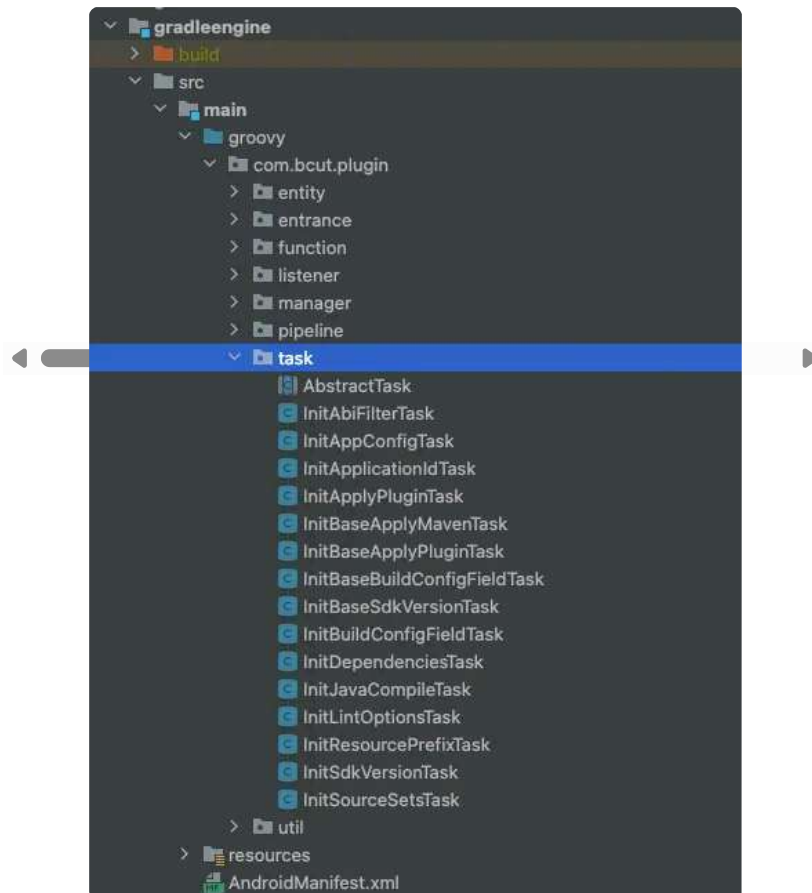
    @Override
    void input(Object ob) {
        super.input(ob)
    }

    @Override
    void doTask() {
        super.doTask()
        System.out.println("GradleEngine || InitBaseSdkVersionTask doTa
PluginInfoManager.getInstance().getProject().android {
    compileSdkVersion BaseConstant.compileSdkVersion
    buildToolsVersion BaseConstant.buildToolsVersion
}
PluginInfoManager.getInstance().getProject().android.defaultConfig {
    minSdkVersion BaseConstant.minSdkVersion
    targetSdkVersion BaseConstant.targetSdkVersion
```



```
    }  
}  
  
@Override  
Object output() {  
    return super.output()  
}  
}
```

GradleEngine核心task代码目录:



备注:

如果对于Gradle不是很熟悉的同学,可以简单的在项目里头,新建gradle文件,提供一些闭包实现、基础配置信息、组件版本信息等等。

GradleEngine的更多介绍请见第九章“工程管理介绍”的内容。

07 老项目组件化落地方法论

前面五个章节,主要讲解了单一工程&模块化&组件化的开发模式、组件化通用化架构和组件化开发规范,接下来分享的是落地方法论。

组件化重构对于比较大型和复杂的商业项目而言难度较大，具体表现如下：

由于历史比较久远，很多代码都运行一年甚至两三年了，各个业务相互交叉耦合严重，文档缺失，不敢随意修改，否则会牵一发而动全身，影响现有业务的正常运行

组件化重构周期较长，业务迭代和组件化重构是并行的，新的需求如何最高效高质量的合入新的架构？

组件化重构影响范围是所有业务，测试需要全量测试，工作量较大

当评估完项目重构的难度之后，就需要根据实际的问题，给出解决方案：

梳理整体业务，聚类问题，然后确定技术方向和技术方案

梳理项目依赖方和技术难点，哪些技术点需要提前调研

前面两点确定好之后，再将大的问题分类继续更细粒度的拆分成小问题以及对应的解决方案

将整体任务拆分完之后，输出工时和Milestone

组件化重构是一个比较大型的技术需求，需要耗费较多的人力，这时候就要考虑ROI了，需要输出组件化重构目标产出：

上层对下层垂直化和按需依赖减少网状式依赖，尽可能减少因为底层的修改影响全部业务

迭代效率能提高多少？维护成本能降低多少？，具体衡量指标这里就不详细阐述了

抽离、重构和输出基础功能组件、业务组件和平台级组件，提高代码复用性、为输出平台化组件提供可扩展性框架和基建

整体组件化，每个组件都可以按需编译，每个业务模块独立编译和运行，编译效率能提高多少？

08 具体实施

第一步：梳理以前的架构和业务，如下图是必剪Android端以前的架构（没完全列举）



1. 项目架构介绍

整个项目分为四层，从上往下分别是：

- APP Module Layer：业务层，例如必剪APP大首页、bgm模块、素材集市模块、录屏模块等等；
- Common Layer：基础业务服务层，顾名思义就是一些基础业务组件，包含了主编辑器、asr、年报等；
- Framework Layer：自研基础功能组件层，例如上图中的上传组件、数据库组件、下载组件等等；
- Libraries Layer：包含各种开源库、主站基架和第三方SDK

2. 架构问题

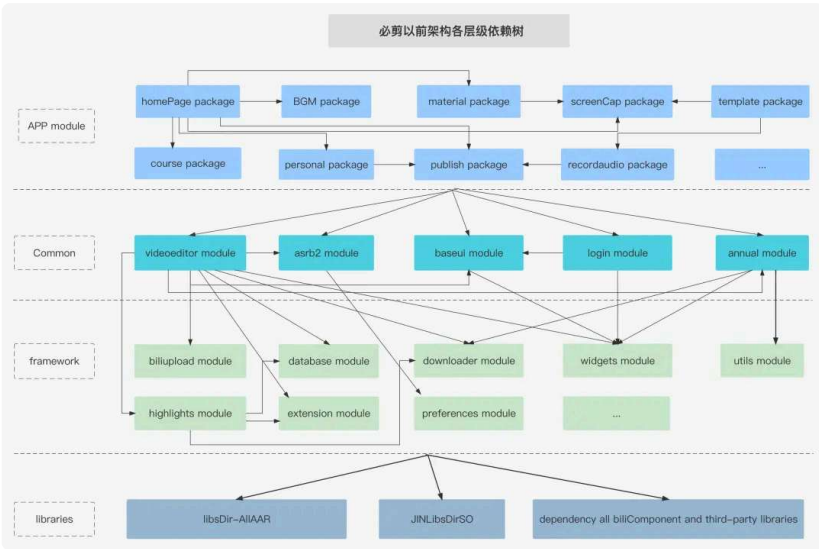
通过以上的架构图，可以看出来，架构分层没什么大问题，但每一层都存在很多问题，比如：

- APP Module Layer存在的问题是每个业务模块过于臃肿，以包的形式进行组织，模块之间存在横向依赖和耦合，灵活性差。代码阅读性差、新人学习成本高、迭代和维护成本大，经常引起功能衰退
- Common Layer包含了垂直业务模块—主编辑器，且存在很多横向依赖
- Libraries Layer的结构特别不合理，包含了所有的第三方SDK、JIN、开源库、主站基架。上层的所有组件和package直接遍历依赖所有的组件

那个时期，各业务技术方案不统一，冗余&重复代码充斥项目的各个角落，大家只不过是不停地往上堆砌代码添加新功能罢了，迭代和维护成本已经出现了较严重的问题。

第二步：梳理项目依赖树

1. 项目整体依赖树



2. 依赖树整体问题

通过以上的依赖树图，可以看出，架构各层依赖凌乱，同一层存在过多的横向依赖，各种耦合且没有内聚。librarys以上的架构，都是直接依赖整个libraries Layer，这种依赖方式太重，无法组件化和输出基础组件。

3. librariesLayer 依赖树



第三步：问题聚类，给出技术方案

架构准则

只有上层的组件才能依赖下层组件，不能反向依赖，会出现循环依赖的情况
同一层之间的组件不能相互依赖，为了组件之间彻底解耦

每一层级的问题聚类和解决方案

libraries Layer

1. 该层级职责边界

自研基础功能组件（跟业务无关）、第三方SDK module封装

maven统一管理

2. 该层级问题描述

结构不合理，上层如果直接依赖libraries Layer就会导致上层的各种组件、业务模块就算拆分了更细粒度和抽离公共模块，也无法实现平台化，因为别的平台不会因为使用一个基础组件，需要依赖这么多底层组件库

3. 解决方案

libraries Layer的结构重整，打破原有不合理的结构，每个基础库都是单独module组件且上传至maven

上层按需依赖

明确每个module的代码提交权限、Owner权限以及review机制

创建组件仓库独立管理

framework Layer

1. 该层级职责边界

这一层提供两种组件，分别是

一种是对更下层的SDK或开源库进行二次封装、定制和代理。比如BCutFresco, downloader、player、tts、asr等组件，

另一种是根据上层业务抽象和实现基础能力，比如主编辑器引擎、guide、database组件等

2. 该层级问题描述

framework Layer 组件库之前有横向依赖，从架构的合理性来说，同一层的组件应该尽可能避免耦合和相互依赖

3. 解决方案

筛选出相互依赖的组件，根据功能边界和高内聚，进行局部重构，部分组件下沉至更底层libraries Layer；跟业务强关联的组件上浮至common Layer

common Layer

1. 该层级职责边界

支撑上层业务组件运行的基础业务服务，对一些系统通用的业务能力进行封装组件。例如主编辑器引擎服务池，虚拟形象服务池，鬼畜服务池、图文服务池等

给业务定制和关联的组件，比如模板下载、草稿管理、导出组件等等

2. 该层级问题描述

包含业务模块module

有些组件是跟业务无关的功能组件

3. 解决方案

业务模块module上浮至APP Module Layer，业务无关的功能组件下沉至framework Layer

APP module

1. 该层级职责边界

该层的组件就是我们真正的业务组件了，我们通常按照功能模块来划分业务组件，比如主编辑器模块、虚拟形象模块、轻图文模块、鬼畜模块、配音模块、BGM模块

2. 该层级问题描述

业务模块之前存在严重的耦合，模块过于臃肿

3. 解决方案

梳理业务，定义好业务边界，输出模块拆分计划

业务代码解耦和内聚

第四步：任务拆解

老项目实施组件化，会比一个新项目开始组件化难度会大些，主要是对于老业务的影响和新业务的合入。如下是**必剪老项目组件化的一些实践经验**，供大家参考

整体思路

完成基础能力建设，gradle统一管理插件、创建组件仓库、组件版本管理插件、支持一键切换集成模式和非集成模式插件、按需编译插件等

从底层往上层进行优化和重构，每一层需要做的事情都差不多，梳理现有的代码架构和业务架构，解耦&内聚，资源迁移和重命名、设计暴露API、实现功能。根据约定好的分层，放在对应的层级里

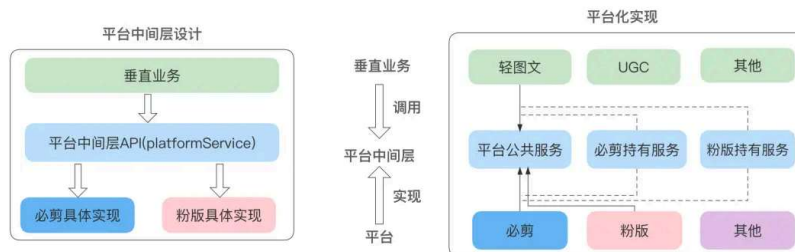
具体任务拆解

名称	任务项信息
跟业务代码无关的基础管理能力	1、gradle 统一管理插件 2、创建组件仓库 3、组件版本管理插件，版本号、本地和 maven 发布 4、支持一键切换集成模式和非集成模式插件 5、按需编译插件 6、支持底层组件切换源码和 maven 依赖
组件化架构基础能力	1、路由框架选型 2、组件生命周期管理
输出代码架构图	根据业务架构和代码梳理，重新输出组件化代码架构图，包含如下 1、架构层级以及每一层的职能边界 2、对外暴露 API 设计规范和实现范例
Libraries Layer	1、梳理该层所有的组件库，查看依赖树，尽量不能横向依赖，根据职能边界进行局部重构，上浮部分组件至 framework Layer 甚至是更上层 2、每个组件都上传至远程组件仓库
framework Layer	同上
common Layer	1、判断哪些服务是大业务模块，不是基础服务，则需要对该模块进行业务梳理和拆分，上浮至业务层，然后暴露服务接口 2、判断哪些服务是跟业务无关的业务组件，则需要下沉至 framework Layer 或者 Libraries Layer
app module Layer	根据以上已经完成的业务梳理，对模块进行拆分、解耦和下沉

09 组件化改造结果

必剪组件化改造后架构图





项目架构介绍

图中可以看到，从上往下分为5层：APP壳工程、业务层、基础业务服务层、自研组件层，商业&部门&主站&开源SDK层

上往下分别是：

1. APP壳工程

壳工程依赖了需要集成的业务组件，主要是一些配置文件，没有任何代码逻辑，根据你的需要选择集成你的业务组件，不同的业务组件就组成了不同的APP

2. 业务层

该层的组件就是我们真正的业务组件了，我们通常按照功能模块来划分业务组件，比如主编辑器模块、虚拟形象模块、轻图文模块、鬼畜模块、配音模块、BGM模块等等，这里的每个业务组件都是一个小的APP，它必须可以单独编译，单独打包成APK在手机上运行。

3. 基础业务服务层

支撑上层业务组件运行的基础业务服务，对一些系统通用的业务能力进行封装组件。对于平台业务，抽象了平台级中间层协议，基础业务可支持平台化（复用至别的部门），对于各垂直业务抽象了暴露服务API。还实现和封装了主编辑器引擎服务池，虚拟形象服务池，鬼畜服务池、图文服务池等，其他业务只要集成对应的组件就可以使用该组件提供的能力集；例如主编剪辑引擎平台服务组件，封装和实现了基础剪辑能力。

4. 自研组件层

Framework Layer:这一层组件提供两种组件，一种是对更下层的SDK或开源库进行二次封装、定制和代理。比如BCutFresco, downloader、player、tts、asr等组件。为什么要对底层SDK进行封装或代理，代理的好处是替换底层SDK，上层业务可以做到无感知。为什么要对底层SDK机型修改定制，比如Fresco，Fresco RGB565配置不生效的兼容性修改；另一种组件是根据上层业务抽象和实现基础能力，比如主编辑器引擎组件、guide、database组件等

Libraries Layer:自研基础功能组件和第三方SDK进行module封装上传至maven统一管理

5. 商业&部门&主站&开源SDK层

顾名思义，就是商业SDK、部门SDK、主站组件和开源组件容器层（每个SDK都进行maven统一管理）

工程管理介绍

1. Gradle通用配置统一管理

1.1 能力集

gradle基础配置信息，比如每个组件依赖的通用插件

```
kotlin-android
kotlin-android-extensions
kotlin-kapt
```

BuildConfig注入业务基础信息

Android系统相关SDK版本

```
targetSdkVersion
minSdkVersion
compileSdkVersion
buildToolsVersion
```

APP主工程集成打包自动依赖所有业务组件

Java版本号

lintOptions

resourcePrefix

sourceSets

编译耗时统计

打包管理

其他

1.2 实现原理

Gradle的统一管理实现方案有两种

第一种是在项目中创建BuildConfig配置文件，把这些公共的配置项定义为共享变量，提供一些闭包封装配置操作，源码如下：

```
ext {
    /**
     * Android SDK公共变量
     */
    android = [
        compileSdkVersion: 31,
        buildToolsVersion: "31.0.0",
        minSdkVersion      : 21,
```

```

        targetSdkVersion : 28
    ]

    /**
     * SDK maven版本号
     */
    masterlib = [
        "${project.groupId}.accountinfo" : "${project.groupId}.accountinfo:${project.version}",
        "${project.groupId}.accountsui" : "${project.groupId}.accountsui:${project.version}",
        "${project.groupId}.updater" : "${project.groupId}.updater:${project.version}",
        "${project.groupId}.neuron" : "${project.groupId}.neuron:${project.version}",
        "${project.groupId}.blkv" : "${project.groupId}.blkv:${project.version}",
        "${project.groupId}.foundation" : "${project.groupId}.foundation:${project.version}",
        "${project.groupId}.downloader" : "${project.groupId}.downloader:${project.version}",
        "${project.groupId}.router_extras" : "${project.groupId}.router_extras:${project.version}",
        "${project.groupId}.router_compat" : "${project.groupId}.router_compat:${project.version}",
        "${project.groupId}.bconfig" : "${project.groupId}.bconfig:${project.version}",
        "${project.groupId}.blconfig" : "${project.groupId}.blconfig:${project.version}"
    ]

    /**
     * 设置Router依赖闭包
     */
    setRouterDependencies = {
        project -> {
            project.dependencies {
                kapt "${project.groupId}.router_apt:${project.version}";
                implementation "${project.groupId}.blrouter_api:${project.version}";
                implementation "${project.groupId}.router_extras:${project.version}";
                implementation "${project.groupId}.router:${project.version}";
                implementation "${project.groupId}.routerbase:${project.version}";
            }
        }
    }
}

```

第一种技术方案实现有如下缺点：

1. 类似的通用配置跟业务无关，不需要暴露给业务开发同学
2. 工程的通用配置、业务配置项、各种插件的依赖、编译耗时统计、library和application动态切换等，代码就会显得比较臃肿。扩展性不高且无法复用至别的业务线

第二种实现方案是自定义Gradle插件，把gradle的所有配置项当成需求进行抽象和实现，提供一个可扩展和平台化的Gradle引擎。

插件入口类：

```

class EngineMain implements Plugin<Project> {
    @Override
    void apply(Project project) {
        PluginInfoManager.getInstance().setProject(project)
        PluginInfoManager.getInstance().initProjectConfig(project)

        if (!project.getName().equals(project.getRootProject().getName())) {
            switch (PluginInfoManager.getInstance().getProjectConfig().projectType) {
                case ProjectTypeDefine.STUDIOMAINPROJECT:
                    initStudioMainProjectGradleCF(project)
                    break
                case ProjectTypeDefine.STUDIOMAVENLIBS:
                    initStudioMavenlibsProjectGradleCF()
                    break
                case ProjectTypeDefine.COMMPROJECT:

```

```

        initCommLibGradleCF()
        break
    default:
        println("&quot;请指定项目类型，ProjectTypeDefine&quot;");
        break
    }
}
}
}
}

```

抽象：每个配置项，抽象成一个task，且task都有对应的基类，比如apply plugin配置，library和application所有需要的apply plugin是不一样的。

但又有共性，那是否可以对此进行抽象呢？答案是可以的，示例代码如下：

```

class InitBaseApplyPluginTask extends AbstractTask<Object> {
    @Override
    void input(Object ob) {
        super.input(ob)
    }

    @Override
    void doTask() {
        super.doTask()
        initLibraryPlugin()
        initkotlinPlugin()
        initASMPPlugin()
    }

    @Override
    Object output() {
        return super.output()
    }

    protected void initLibraryPlugin(){
        PluginInfoManager.getInstance().getProject().apply plugin:
        BaseConfigDefine.PluginDefine.PLUGINMAP.get("&quot;library&quot;");
    }

    protected void initASMPPlugin(){
        PluginInfoManager.getInstance().getProject().apply plugin:
        BaseConfigDefine.PluginDefine.PLUGINMAP.get("&quot;asmonitorplugin&quot;");
    }

    protected void initkotlinPlugin() {
        PluginInfoManager.getInstance().getProject().apply plugin:
        BaseConfigDefine.PluginDefine.PLUGINMAP.get("&quot;kotlinandroid&quot;");
        PluginInfoManager.getInstance().getProject().apply plugin:
        BaseConfigDefine.PluginDefine.PLUGINMAP.get("&quot;kotlinandroidexten");
        PluginInfoManager.getInstance().getProject().apply plugin:
        BaseConfigDefine.PluginDefine.PLUGINMAP.get("&quot;kotlinlinkapt&quot;");
    }
}

class InitApplyPluginTask extends InitBaseApplyPluginTask{

    @Override
    void input(Object ob) {
        super.input(ob)
    }

    @Override
    void doTask() {
        PluginInfoManager.getInstance().getProject().getRootProject().ext.ap

```

```

        if (v.contains(PluginInfoManager.getInstance().getProject().getN
            if (!PluginInfoManager.getInstance().getProjectConfig().getI
                initApplicationPlugin()
            } else {
                initLibraryPlugin()
                initkotlinPlugin()
            }
        }
    }
}

@Override
Object output() {
    return super.output()
}

protected void initApplicationPlugin() {
    PluginInfoManager.getInstance().getProject().apply plugin: BaseCon
    initkotlinPlugin()
    PluginInfoManager.getInstance().getProject().apply plugin: BaseConfi
    PluginInfoManager.getInstance().getProject().apply plugin: BaseConfi
}

}

```

可扩展性

不同类型的module配置信息，主工程、业务模块（library和application自动切换）、基础组件，示例代码如下：

```

private void initStudioMainProjectGradleCF(Project project) {
    println("&quot;GradleEngine || initStudioMainProjectGradleCF&quot;");

    /**
     * 空壳APP Module（主工程）
     */
    if (project.getName().equals(project.getRootProject().ext.mainModuleN
        println("&quot;GradleEngine || initStudioMainProjectGradleCF mainN
        PluginInfoManager.getInstance().executeAppConfigPipeline()
        PluginInfoManager.getInstance().executeBusinessModulePipeline()
    } else {
        /**
         * 业务modules
         */
        project.getRootProject().ext.app_business_module.each { k, v -&gt;
            if (v.contains(project.getName())) {
                println("&quot;GradleEngine || initStudioMainProjectGradle
                PluginInfoManager.getInstance().executeBusinessModulePipe
                targeted = true
                return
            }
        }
    }
    /**
     * 基础组件
     */
    println("&quot;GradleEngine || initStudioMainProjectGradleCF 其他
    PluginInfoManager.getInstance().executeBaseLibLayerPipeline()

}
}

```

工作流扩展性，工作流的task支持任意拔插和组合，支持任意扩展和修改，示例代码如下：

```

/**
 * 业务组件
 */
public void executeBusinessModulePipeline() {
    System.out.println(""GradleEngine || executeBusinessModulePipeline8
    BusinessModulePipeline.builder().setInitApplypluginTask(getBusinessAbst
        .setInitApplicationIdTask(getBusinessAbstractTask(TaskTypeDe
        .setInitBaseApplyMavenTask(getLibModuleAbstractTask(BaseLibT
        .setInitSdkersionTask(getBusinessAbstractTask(TaskTypeDefine
        .setInitAbiFilterTask(getBusinessAbstractTask(TaskTypeDefine
        .setInitBuildConfigFieldTask(getBusinessAbstractTask(TaskTyp
        .setInitMainModuleDependenciesTask(getBusinessAbstractTask(T
        .setInitLintOptionsTask(getBusinessAbstractTask(TaskTypeDefi
        .setInitJavaCompileTask(getBusinessAbstractTask(TaskTypeDefi
        .setInitSourceSetsTask(getBusinessAbstractTask(TaskTypeDefin
        .setInitResourcePrefix(getBusinessAbstractTask(TaskTypeDefin
        .execute()

    }

/**
 * 基础组件
 */
void executeBaseLibLayerPipeline() {
    LibModulePipeline.builder().setInitBaseApplypluginTask(getLibModuleAbstractT
        .setInitBaseApplyMavenTask(getLibModuleAbstractTask(BaseLibT
        .setInitBaseBuildConfigField(getLibModuleAbstractTask(BaseLi
        .setInitBasesdkVersionTask(getLibModuleAbstractTask(BaseLibT
        .setInitBaseJavaCompileTask(getLibModuleAbstractTask(BaseLibT
        .execute()

    }

```

task代码示例

主工程（壳工程module）自动集成所有的业务组件，示例代码如下：

```

class InitDependenciesTask extends AbstractTask<Object> {
    @Override
    void input(Object ob) {
        super.input(ob)
    }

    @Override
    void doTask() {
        super.doTask()
        PluginInfoManager.getInstance().getProject().dependencies {
            if (PluginInfoManager.getInstance().getProject().getName().equals(
PluginInfoManager.getInstance().getProject().getRootProject().ext.mainModule
            if (PluginInfoManager.getInstance().projectConfig.getIntegrated()) {
                /**
                 * 依赖各个业务module
                 */
                PluginInfoManager.getInstance().getProject().getRootProject().e
                    implementation PluginInfoManager.getInstance().getProject()
                }
            }
        }
    }
}

@Override
Object output() {
    return super.output()
}

```

}

2. 快编构建系统

能力集

编译耗时统计和分析

按需编译

全量编译10分钟降低至4分钟

增量编译3分钟降低至1.2分钟

每个同学每天编译耗时预计可以从50分钟降低至25分钟

编译耗时统计分析，效果图如下：



编译耗时统计分析实现原理比较简单，核心是主工程project实现

TaskExecutionListener, BuildListener, 核心代码如下:

```
class BuildTimeListener implements TaskExecutionListener, BuildListener {
    private final String BUILD_LOG_FILE_DIR
    private long taskStartTime
    private long buildStartTime
    private SimpleDateFormat dateD = new SimpleDateFormat("&quot;yyyy_MM_dd&quot;")
    private SimpleDateFormat dateS = new SimpleDateFormat("&quot;HH_mm_ss&quot;")
    private HashMap<String, Long> taskInfoHas = new HashMap<>();

    BuildTimeListener(Project project) {
        buildStartTime = System.currentTimeMillis()
        /**
         * 获取项目的绝对路径
         */
        String[] rootDirList = project.getRootDir().getAbsolutePath().split(
            String baseDIR
        /**
         * 根据约定好的目录协议，得到编译耗时文件目录根目录
         */
        if (rootDirList.length > 3) {
            baseDIR = new StringBuilder().append(rootDirList[0]).append("&quot;")
                .append("&quot;").append(rootDirList[2]).append("&quot;/&quot;").toString()
        } else if (rootDirList.length > 1) {
            baseDIR = new StringBuilder().append(rootDirList[0]).append("&quot;")
                .append("&quot;").toString()
        }
    }
}
```

```

    }
    BUILD_LOG_FILE_DIR = baseDIR + "&quot;build_history&quot;;
    if (!new File(BUILD_LOG_FILE_DIR).exists()) {
        new File(BUILD_LOG_FILE_DIR).mkdir()
    }
}

@Override
void buildFinished(BuildResult result) {
    StringBuilder sb = new StringBuilder()
    long buildCost = System.currentTimeMillis() - buildStartTime
    ArrayList<Map.Entry<String, Long>> arrayList = sortMap(t
    for (int i = 0; i < arrayList.size(); i++) {
        if (i == 10) {
            break
        }
        Map.Entry<String, Long> entry = arrayList.get(i)
        sb.append(entry.getKey() + "&quot;=&quot; + entry.getValue() + &c
    }
    sb.append("&quot;totalDuration=&quot; + buildCost + "&quot;\n&quot;")
    writeFile(sb.toString())
}

private ArrayList<Map.Entry<String, Long>> sortMap(HashMap<L
Set<Map.Entry<String, Long>> set = oldhMap.entrySet();
ArrayList<Map.Entry<String, Long>> arrayList = new Array
Collections.sort(arrayList, new Comparator<Map.Entry<String, L
    @Override
    int compare(Map.Entry<String, Long> arg0,
        Map.Entry<String, Long> arg1) {
        return arg1.getValue() - arg0.getValue();
    }
}

return arrayList
}

@Override
void beforeExecute(Task task) {
    taskStartTime = System.currentTimeMillis();
}

@Override
void afterExecute(Task task, TaskState state) {
    long cost = System.currentTimeMillis() - taskStartTime
    taskInfoHas.put(task.getPath(), cost)
}

private void writeFile(String text) {
    String basePath = BUILD_LOG_FILE_DIR + "&quot;/&quot; + dateD.format(
    File dateDir = new File(basePath)
    if (!dateDir.exists()) {
        dateDir.mkdir()
    }
    File file = new File(basePath + "&quot;/&quot; + dateS.format(new Dat
    file.write(text, true)
}
}

```

快编插件（按需编译）

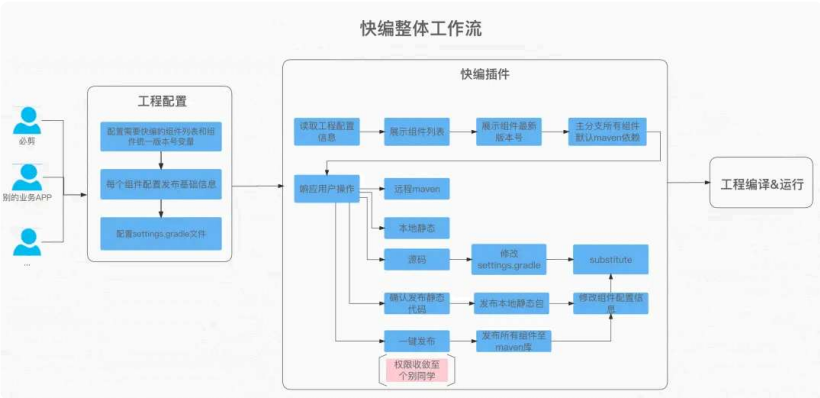
插件UI交互

必剪快速编译

模块名称	依赖方式	版本号	静态发布
bgm	☒ 远程Maven ☐ 本地静态 ☐ 源码	8.006	确认发布
course	☐ 远程Maven ☐ 本地静态 ☒ 源码		确认发布
cover	☐ 远程Maven ☒ 本地静态 ☐ 源码	8.007	确认发布
dubbing	☒ 远程Maven ☐ 本地静态 ☐ 源码		确认发布
guichu	☒ 远程Maven ☐ 本地静态 ☐ 源码	8.006	确认发布
homepage	☒ 远程Maven ☐ 本地静态 ☐ 源码	8.006	确认发布
mainvideoeditor	☒ 远程Maven ☐ 本地静态 ☐ 源码	8.006	确认发布
material	☒ 远程Maven ☐ 本地静态 ☐ 源码	8.006	确认发布
personal	☒ 远程Maven ☐ 本地静态 ☐ 源码	8.006	确认发布
publish	☒ 远程Maven ☐ 本地静态 ☐ 源码	8.006	确认发布
tuwen	☒ 远程Maven ☐ 本地静态 ☐ 源码	8.006	确认发布
virtualidol	☒ 远程Maven ☐ 本地静态 ☐ 源码	8.006	确认发布

更多组件，这里就不一一展示了

主分支一键发布和更新所有组件（只有个别同学才有权限）



快编实现原理

按需编译的前置条件是工程需要组件化，每个组件做到单一职责。接下来就是提供组件发布和依赖的自动化脚本或插件了

发布

A 远程发布：

一般是主分支的代码，才需要发布至远程仓库。然后发布的时机，可以根据自己部门或公司的实际情况进行自定义，比如CI阶段代码合并至主分支，增量发布对应的组件或者本地提供Androidstudio插件，进入总包阶段，每一天自动全量发布组件

B 本地发布：

开发阶段自己的工作分支，组件可以发布至本地，自己使用，不影响别的开发同学或业务线

自动依赖

当快编插件触发操作时，快编插件需要动态修改工程对应的配置文件，比如将bgm改成源码依赖，通过gradle substitute API将进行依赖替换比如substitute module(bgm) using project(bgm)，修改settings文件，include bgm module;

动态修改组件版本号

注意事项

组件间的资源文件尽量不要跨组件依赖，如果出现了跨组件依赖，各组件进行了依赖替换的时候，编译会报Android Resource linking failed

未来优化项

组件做到按需编译后，全量和增量编译效率虽然都有了很大的提升，但还有继续优化的空间，下一个目标是实现秒编，代码和资源增量，推送至设备进行热部署

10 总结

组件化的基础思想是相通的，遵守组件化的基本规范，搭建组件化架构，再根据代码设计七大原则，抽离或下沉高内聚、低耦合、高复用的业务和功能组件。必剪的组件化改造是跟着迭代逐渐完善的，并且以后肯定也会随着业务和代码的变动不断的进行优化，这会是一个持续的过程。这次组件化改造的过程中涉及了太多的内容，其中很多点都能拿出来单独写一篇文章，比如快编的完整架构和实现细节，自定义Gradle插件-GradleEngine，gradle构建流程，AndroidStudio插件开发等等，由于篇幅有限并不能在文中一一详述，希望后续有时间可以分享更多的技术文章跟大家一起学习和交流。

以上是今天的分享内容，如果你有什么想法或疑问，欢迎大家在留言区与我们互动，如果喜欢本期内容的话，请给我们点个赞吧！



哔哩哔哩技术

微信扫描二维码，关注我的公众号

B站 必剪

分享至



投诉或建议

评论 2 赞与转发

最热 | 最新



这里需要一条查重率0%的评论



Charlottener LV5

好详细

2024-08-01 00:35 回复



无向白 LV6

2022-10-14 14:04 回复

没有更多评论